

LLMs Under Rules: Mode 2 of the CKS Labor Allocation Framework as a Standalone Architectural Specification

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in "Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems" (Li, April 2026).

Author: Wenxin Li (Independent Researcher)

ORCID: 0009-0004-8065-3235

Date: May 5, 2026

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Abstract

The Coordination Knowledge Substrate (CKS) pattern's labor allocation framework names three modes by which coordination work can be performed: humans directly (Mode 1), LLMs under human-authored orchestration rules (Mode 2), and stable cells largely automating execution under those rules (Mode 3). This note formalizes Mode 2 as a standalone architectural specification: the labor-mode realization of the AI-as-substrate-mediator commitment, in which LLM token-output non-determinism is bounded by rule-governance such that substrate writes remain rule-conformant. The note defines Mode 2 as four operational components (rule-governed execution, cell-bounded operation, mediator-properties operation, cell-with-rule writer attribution); states what Mode 2 does not claim; distinguishes Mode 2 from four adjacent labor patterns commonly conflated with it; names ten failure modes by which an implementation can violate Mode 2; and provides an operational test. Mode 2 is the architecturally most-specified of the three labor modes; its standalone formalization is what forecloses the autonomous-agent and prompt-engineering drifts that otherwise consume nominally rule-governed AI architectures.

1. Why Mode 2 needs to be formalized as standalone

The parent foundational note A1.12 commits to the labor allocation framework as a single architectural property — three modes subject to the same human-governed authority structure. The integrating-frame note A2.68 names Mode 2 at the framework level. Sibling note A2.69 formalizes Mode 1 in standalone form; sibling note A2.71 will do the same for Mode 3. This note does the same for Mode 2.

Mode 2 is the labor-mode realization of A1.04 (AI-as-substrate-mediator). A1.04 specifies that the LLM operates as substrate mediator with the five properties decomposed in A2.18–A2.23; A2.70 specifies how that commitment manifests when expressed as a labor mode. Motivating cases are decision-support cells, content-extraction cells, summarization cells — in each, the LLM performs labor (reading substrate, reasoning over content, writing substrate) under human-authored rules that determine the labor's shape.

The strategic motivation for the standalone treatment is that Mode 2 is the most commercially-pressured of the three modes. Audiences understand "AI agent" or "autonomous AI" more readily than "LLM as substrate mediator under orchestration rules," and the simpler framings drift toward architectures that violate Mode 2's commitments — by removing rule-governance, by allowing the LLM to make decisions about substrate content based on its own judgment, or by placing

orchestration logic in the LLM rather than in rules. Naming Mode 2 as a standalone specification with explicit failure modes is what makes those drifts visible as drifts rather than as unremarkable variations. As public prior art, the standalone formalization narrows the territory in which any party could later claim novel invention of an LLM-under-rule coordination architecture.

2. The Mode 2 commitment, defined precisely

A system implements Mode 2 if and only if all four of the following operational components hold for every Mode 2 cell execution.

(a) Rule-governed execution. Every Mode 2 cell execution operates under a specific orchestration rule authored under A2.04's rule-authoring-as-governance moment, with rule-conformance per Guarantee B per A2.58. The rule specifies what the cell can read, what it can write, and what constitutes a rule-conformant output. The architectural commitment is that the LLM's labor is rule-bounded: identical substrate state and identical rule produce equivalent cell behavior at the substrate-write level, even if specific LLM token outputs vary within the rule's allowed structure (per A2.62 category (a)). LLM operation outside a specific rule's scope is not Mode 2 — it is either autonomous LLM operation, which the mediator commitment per A1.04 forbids, or a different labor pattern (e.g., a Pattern A LLM input feeding into a cell, per A1.16).

This component operationally distinguishes Mode 2 from autonomous-agent patterns. Autonomous AI agents place authority in the LLM — the LLM decides what to do, what to write, what action to take. Mode 2 places authority in the rule, and through the rule, in the human who authored it; the rule itself is human-authored substrate content per A2.46's Category 4 source-of-truth commitment. The LLM decides only at the level the rule permits.

(b) Cell-bounded operation. Mode 2 LLM operation occurs within cells per A1.02's substrate-cell boundary. The cell is the locus where the LLM operates: it reads substrate state through A2.10's substrate→cell boundary crossings and writes substrate state through A2.10's cell→substrate boundary crossings. LLM operation outside cells — direct LLM writes to substrate, LLM-to-LLM communication that bypasses substrate, LLM access to state not substrate-resident — is not Mode 2. Without the substrate-cell boundary, Mode 2 would have no architectural locus.

(c) Mediator-properties operation. Mode 2 satisfies the five mediator properties decomposed from A1.04 as A2.19–A2.23: the LLM reads substrate as primary source of state (Property A); writes under orchestration rules (Property B); does not hold substrate-relevant state outside substrate (Property C); does not exercise authority over substrate content (Property D); and produces outputs that affect substrate state with attribution recorded (Property E). The five properties are not independent claims here — A1.04 defends the commitment and A2.18–A2.23 decompose it. This component states that the five properties are operationally satisfied during every Mode 2 cell execution, not only as a design-time commitment.

(d) Cell-with-rule writer attribution. Mode 2 substrate writes carry attribution identifying both the cell that performed the write and the orchestration rule under which the cell operated. This is the writer-attribution category A2.37 names for Mode 2, and it operationally distinguishes Mode 2 substrate content from Mode 1 content (direct human attribution) and from Mode 3 content (stable-cell-with-rule attribution, distinguishable from Mode 2 by the rule's operational stability rather than by the attribution mechanism). The architectural commitment is that the attribution is inspectable

and identifies the cell and the rule, not merely "the LLM" or "the system."

This component operationally distinguishes Mode 2 from Mode 1 at the substrate level. Mode 1 writes are attributed to the human writer directly; the human is responsible for the specific content. Mode 2 writes are attributed to the cell-with-rule; the human's responsibility is at the rule, not at the specific content the rule produced when applied to a specific substrate state by a specific LLM execution. The two attributions name two operationally different relationships between writer and content, and the framework relies on the distinction to make labor mode itself an inspectable property of substrate content.

The four components together define Mode 2 architecturally. A system that supports all four has Mode 2; a system that fails any one of them does not, regardless of how operationally LLM-like its behavior appears.

3. What Mode 2 does NOT claim

The standalone treatment is bounded.

It does not claim LLM determinism. Mode 2 operates with allowed non-determinism per A2.62 category (a): LLM token outputs may vary across executions even with identical inputs. The architectural commitment is to rule-governance bounding the variation per Guarantee B per A2.58 such that substrate writes are rule-conformant — not to the LLM producing identical outputs.

It does not claim Mode 2 is preferable to Mode 1 or Mode 3. The framework is mode-agnostic at the architectural-commitment level; A2.74 treats progression dynamics separately.

It does not specify which LLM technologies to use. Per A1.05's tool-agnosticism commitment, Mode 2 may be instantiated with various LLM technologies, provided the five mediator properties hold for whichever is used.

It does not foreclose deployment-level features. Tools for cell authoring, rule authoring, LLM integration, or output review are deployment choices and are not part of Mode 2's specification.

It does not require Mode 2 to be exercised in every deployment. A deployment may operate entirely in Mode 1. The architectural commitment is that Mode 2 is *available*, not that it is *exercised*.

It does not require all LLMs in the deployment to operate in Mode 2. Adjacent components per A1.16 may include LLMs operating as Pattern A inputs, Pattern B derivative views, or Pattern C separate concerns; Mode 2's commitment applies to LLMs operating *within* Mode 2.

4. What Mode 2 is NOT

Four adjacent labor patterns are commonly conflated with Mode 2; each is operationally distinct.

Not autonomous AI agents. Autonomous AI agents operate with broad autonomy: making decisions, taking actions, writing to systems without architectural rule-governance. The LLM in Mode 2 is bounded by orchestration rules per A2.20, does not exercise authority per A2.22, and is mediator, not agent. Implementations that present autonomous-agent behavior as Mode 2 — typically by gesturing at "rules" that are in fact prompt templates without the architectural status A2.46 requires — violate component (a) of section 2 and the mediator commitment of A1.04. The drift toward autonomous-agent

patterns is the single most consequential drift Mode 2's standalone specification forecloses, because it is the drift most commercially attractive in deployments under pressure to deliver "agentic AI" capabilities.

Not LLM-driven workflows. LLM-driven workflows are operational patterns in which an LLM orchestrates work across multiple steps — choosing which step to execute next, routing between branches, selecting actions. Mode 2 is different: the LLM operates within a single cell under a single specific rule; orchestration logic lives in rules per A2.04 and in substrate per A2.46, not in the LLM's reasoning. Implementations that place orchestration logic in the LLM violate Property B per A2.20.

Not prompt engineering. Prompt engineering is the practice of crafting inputs to LLMs to elicit desired outputs. Mode 2 may include prompt content within rule specifications, but Mode 2 is broader: it is the architectural commitment to LLM operation within cells under rules with the four components of section 2 holding. A system whose only Mode 2-ish property is well-crafted prompts misses cell-bounded operation, mediator-properties operation, and writer attribution.

Not LLM-mediated APIs. LLM-mediated APIs are operational patterns where the LLM acts as a natural-language interface to an existing API. Mode 2 is different: the LLM is not API interface; it is substrate mediator. Implementations that conflate the two miss the substrate-cell boundary per A1.02 that Mode 2 operates within.

5. Why Mode 2 is load-bearing for downstream commitments

Mode 2 is load-bearing for several CKS commitments. For the integrating labor allocation framework per A1.12 and A2.68: Mode 2 is one of the three modes; without it, the framework would lack the LLM-bounded labor mode. For A1.04 and the mediator decomposition A2.18–A2.23: Mode 2 is the labor-level realization; without it, the mediator commitment would be aspirational at the labor level. For A1.02: Mode 2 operates within cells; the substrate-cell boundary is what gives Mode 2 architectural coherence. For A2.04: rules authored under A2.04 are what Mode 2 operates under. For the determinism contract per A1.10, specifically Guarantee B per A2.58 and allowed non-determinism category (a) per A2.62: Mode 2 operates with LLM non-determinism bounded to rule-conformance; these specific commitments are what makes Mode 2 with non-deterministic LLMs architecturally coherent. For non-specialist governance per A1.11: non-specialists govern Mode 2 substrate content per the same three rights they exercise over Mode 1 content, and the architectural commitment that authority is mode-independent is what makes the two equivalent.

6. Failure modes that violate Mode 2

Each of the following describes an implementation pattern that fails Mode 2 by removing rule-governance, allowing autonomous LLM operation, or compromising the mediator commitment. The first four are particularly common in commercial AI implementations under pressure to deliver "agentic" or "autonomous" capabilities, and are weighted accordingly.

(a) LLM operation outside rules. The implementation places LLMs within cells but does not require those cells to reference specific orchestration rules. The LLM operates under implicit prompts, general-purpose instructions, or context-dependent reasoning rather than explicit rule scope. Property B per A2.20 fails — the LLM writes are not under orchestration rules in the architectural sense. This is the most common rule-bypass pattern: it preserves the appearance of cell-mediated operation while

removing the rule-governance that distinguishes Mode 2 from autonomous LLM operation. From the substrate side, the writes look like Mode 2 writes; from the architectural side, they have no rule reference and no rule-conformance guarantee.

(b) LLM authority over substrate. The implementation permits the LLM to decide what to write to substrate based on its own judgment, without rule-governance bounding the decision. The LLM evaluates what would be helpful, what the user seems to want, or what aligns with prior content, and writes accordingly. Property D per A2.22 fails — the LLM exercises authority. The architectural consequence is that human governance over Mode 2 content becomes incoherent: the human can override specific LLM-produced content but cannot effectively govern the *shape* of LLM production because there is no rule to govern. Governance reduces to per-element review, which the labor allocation framework is precisely designed to avoid.

(c) LLM-driven orchestration. The implementation places orchestration logic in the LLM rather than in rules per A2.04. The LLM decides which cells to invoke, which rules apply to which contexts, what work-flow to follow. Cells and rules become reactive responses to the LLM's choices. The architectural commitment per A1.04 fails — the LLM is no longer mediator, it is the system's controller. Substrate becomes a record of what the LLM decided to do, rather than the source of truth that determines what cells run and how. This is the failure pattern that produces "agentic" architectures most commonly described in commercial AI deployments.

(d) Mode 2 with rule-bypass. The implementation provides Mode 2 capability nominally — cells, rules, attribution — but includes rule-bypass mechanisms: special-case execution paths, escape hatches for "edge cases," administrator-mode cells that operate without rule-conformance. The architectural commitment is present in form but operationally violated for the cases the bypass covers. This is the failure pattern most resistant to detection at the architectural level because the bypass is typically justified as a pragmatic accommodation rather than as an architectural compromise.

The remaining failure modes are mechanical and well-characterized.

(e) LLM operation outside cells. The implementation permits LLMs to write substrate directly without cell-mediation per A1.02; component (b) of section 2 fails.

(f) LLM state outside substrate. The implementation permits the LLM to maintain state across cell executions outside substrate (in conversation history, in external memory); Property C per A2.21 fails, and LLM non-determinism may accumulate across executions in ways that propagate to substrate.

(g) Unstructured LLM-output-as-substrate-write. The implementation writes raw LLM outputs to substrate without rule-structuring; rule-conformance per Guarantee B per A2.58 fails.

(h) Mode 2 attribution stripping. The implementation does not distinguish Mode 2 writer attribution from Mode 1 or Mode 3 attribution per A2.37; component (d) of section 2 fails.

(i) Implicit rules. The implementation operates "rules" through prompt templates that are not architecturally substrate-resident per A2.46; rule authority becomes implicit or external.

(j) Hidden Mode 1 promotion. The implementation routes some human writes through Mode 2 cells with implicit rules rather than recognizing them as direct human labor; Mode 1's writer attribution per A2.37 is replaced with cell-with-rule attribution, and the framework's mode distinction is compromised.

7. Operational test

A system supports Mode 2 if and only if all of the following are true at all times during the substrate's existence:

1. Every Mode 2 cell execution references a specific orchestration rule, with rule-conformance verifiable per Guarantee B per A2.58.
2. Every Mode 2 cell execution occurs within a cell per A1.02; LLM substrate writes occur through A2.10's boundary crossings within cell scope.
3. Every Mode 2 cell execution satisfies the five mediator properties per A2.19–A2.23: read commitment, write-under-rule commitment, no-state-outside-substrate commitment, no-authority commitment, recorded-with-attribution commitment.
4. Every Mode 2 substrate write carries cell-with-rule attribution per A2.37, distinguishable by inspection from Mode 1 attribution and Mode 3 attribution.
5. Mode 2's LLM non-determinism is bounded to non-substrate output per A2.62 category (a) and Guarantee B per A2.58; substrate writes are rule-conformant even when specific LLM token outputs vary.

A system that fails any of (1)–(5) does not support Mode 2 in the architectural sense, even if LLMs appear to perform coordination work operationally.

8. Why naming Mode 2 as standalone matters

Implementations under commercial pressure to deliver "agentic AI" or "autonomous AI" capabilities consistently drift toward autonomous-agent patterns that violate Mode 2's architectural specification. The drift is steady because autonomous-agent patterns are commercially attractive and operationally simpler in the short term: rules require authoring; autonomy requires only LLM prompting. The downstream consequences manifest as governance failures (Property D fails — the LLM exercises authority), retraceability failures (the rule reference Property B requires is absent), determinism-contract failures (Guarantee B per A2.58 fails because cell behavior is not rule-equivalent across executions), and framework-incompleteness (the three-mode framework collapses into Mode 1 plus autonomous AI, or Mode 1 plus Mode 3, depending on the drift direction).

Naming Mode 2 as a standalone architectural specification gives downstream readers a precise specification of what LLM-under-rule labor architecturally requires. The subsequent note A2.71 specializes Mode 3, which uses the same architectural mechanisms with operational stability adding reduced-attention properties; A2.72 specializes the three architectural properties that make all three modes coherent; A2.73 specializes the labor-vs-authority distinction; A2.74 specializes mode progression dynamics. Together they give the full operational decomposition of A1.12.

Subsequent work that adopts the CKS pattern, extends it, or argues against it should use "Mode 2" in the sense formalized here. Subsequent work that uses the term differently — most commonly to name autonomous-agent or LLM-driven-workflow patterns — is using a different concept, and the difference should be named.

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems*. April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *LLMs Under Rules: Mode 2 of the CKS Labor Allocation Framework as a Standalone Architectural Specification*. Derivation Note. May 5, 2026. ORCID: 0009-0004-8065-3235.